INTEGRATED SYSTEMS ARCHITECTURE

POLITECNICO DI TORINO

DIPARTIMENTO DI ELETTRONICA E TELECOMUNICAZIONI

digital arithmetic General description

Authors:
XU MENGHAN s319887
Abedi Maryam 313156
SHANCHONG SHANG 314324
Milad Rahmani Nezhad s316776

Professor:
Guido Masera

Academic year: 2024-2025

Contents

1 Digital arithmetic and logic synthesizers

1.1 Introduction and background

The bfloat16 (brain floating point) floating-point format is a computer number format occupying 16 bits in computer memory; it represents a wide dynamic range of numeric values by using a floating radix point. This format is a shortened (16-bit) version of the 32-bit IEEE 754 single-precision floating-point format (binary32) with the intent of accelerating machine learning and near-sensor computing. FP16 uses:
1 bit for the positive/negative sign
8 bits for representing the exponent with base 2.
7 bits for the fraction/significand/mantissa, i.e. value after the decimal point.

in the *FPU unit*, we prepare 10 sets data sample as input for the FPU unit which is unzipped from the archive *cvfpu_lite*. After simulation, we import the file in src into Synopsys for synthesis. We used several methods for optimization: compile_ultra, retiming, CSA, PPARCH.

Modern logic synthesizers, such as Synopsys Design Compiler, with the help of ready-to-use block Design Ware(DW), can simplify the behavioral description of arithmetic operations like addition and multiplication. As well, design compiler is able to infer the components from the netlist. Meanwhile, DW includes a parametric multiplier(`DW02_mult`) that can be implemented in either Carry-save(csa) or Parallel-Prefix (pparch). Using commands such as `set_implementation`, the specific architecture of adders, multipliers, and other elements can be defined before compiling the design.

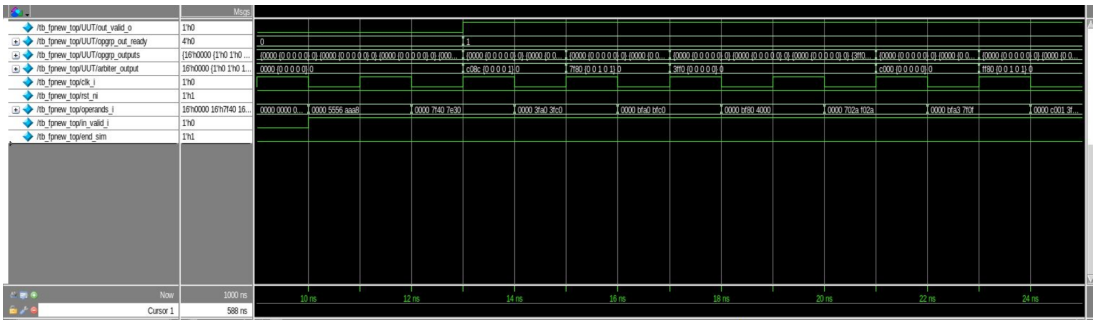

Figure 1: Simulation of the FPU multiplication operation 1 part

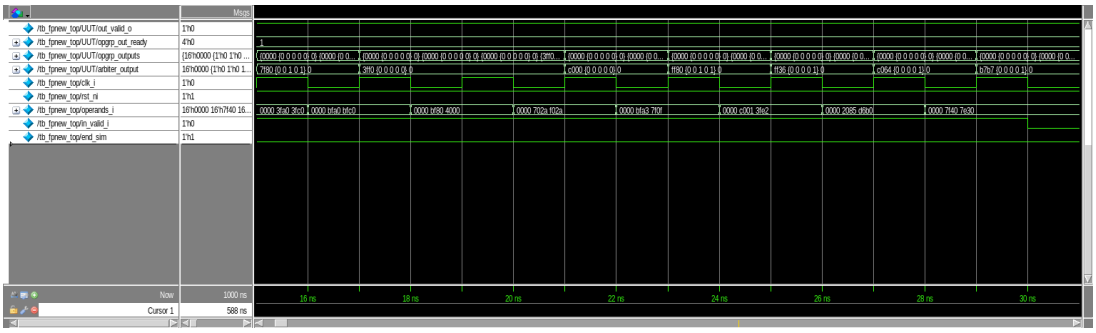

Figure 2: Simulation of the FPU multiplication operation 2 part

1.2 FPU unit

Floating point unit (FPU) further proves this process, by using input in table 1 to test configurations for overflow condition and range limit(7F80 and FF80). Results are generated from fp16.exe, you can find the binary form in sample.txt(in archive.zip-¿fp16_synthesis-¿src-¿samples.txt)

a	b	r
-2.98428×10^{-13} **0xAAA8**	1.4706×10^{13} **0x5556**	-4.375 **0xC08C**
5.8486×10^{37} **0x7E30**	2.55212×10^{38} **0x7F40**	∞ **0x7F80**
1.5 **0x3FC0**	1.25 **0x3FA0**	1.875 **0x3FF0**
-1.5 **0xBFC0**	-1.25 **0xBFA0**	1.875 **0x3FF0**
2 **0x4000**	-1.0 **0xBF80**	-2.0 **0xC000**
-2.1045×10^{29} **0xF02A**	2.1045×10^{29} **0x702A**	$-\infty$ **0xFF80**
1.9008×10^{38} **0x7F0F**	-1.27344 **0xBFA3**	-2.41919×10^{38} **0xFF36**
1.76562 **0x3FE2**	-2.01562 **0xC001**	-3.5625 **0xC064**
-9.6757×10^{13} **0xD6B0**	2.25311×10^{-19} **0x2085**	-2.18153×10^{-5} **0xB7B7**
5.8486×10^{37} **0x7E30**	2.55212×10^{38} **0x7F40**	∞ **0x7F80**

Table 1: Inputs and Outputs of the FPU

Fig 1 and 2 shows a snapshot of the simulation, the result is the same as the way we use the CPP module to calculate, as shown in Table 1

1.3 Synthesis

compile At first synthesis,we use 'compile' command to run Synopsys Design Compiler to synthesizes our FPU unit into optimized design. It can optimize both combinational and sequential designs for area, timing, and power. From the table 2, we can see the clock period becomes 2.87ns (see the figure 3),the area become 2937.438021 μm2 and the maximum frequency is 348.43 MHz, as shown in the table 3. The command file is in Appendix3.1.1, it shows the load modules and delay constraints, finally create timing/area reports and output netlist and constraints files.

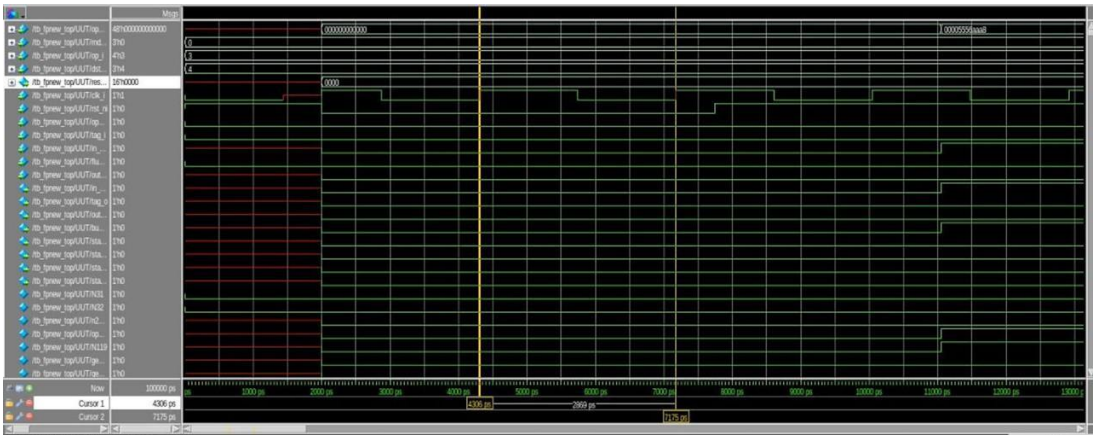

Figure 3: Netlist verification with Compile

The clock cycle 2.87ns as the minimum clock period . When we simulate, we use this data as the time unit. After 10 multiplications, the time cycle is 28.7ns, see the figure4.

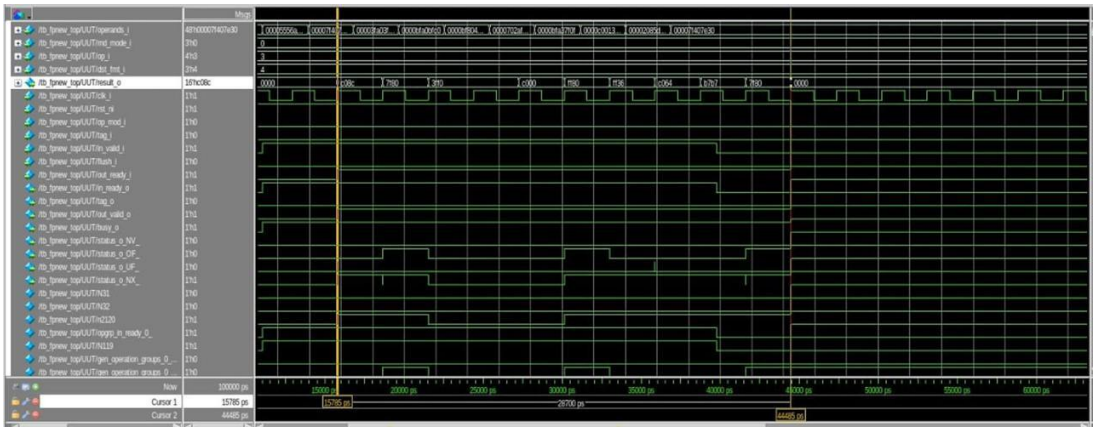

Figure 4: Netlist verification with 10 times multiplications

compile-retiming At second synthesis,we aim to improve the performance by recombining logic gates.The 'optimize-registers' command adds an opportunity for improving circuit timing. It is a circuit optimization technique that aims to shorten the clock cycle or reduce circuit area by moving registers forward or backward in a circuit design. It performs retiming of sequential cells for pipelined designs. From the table 2,we can see the clock period becomes 2.12ns ,the area become 3634.358040 μm2 and the maximum frequency is 471.70 MHz, see the table 3.The command file is in Appendix3.1.2, it include file analyze and compile process, we set the clock period and clock uncertainty as. Timing and Area report are generated and finally the synthesized netlist file(.v) and timing constraint file(.sdc) are output.

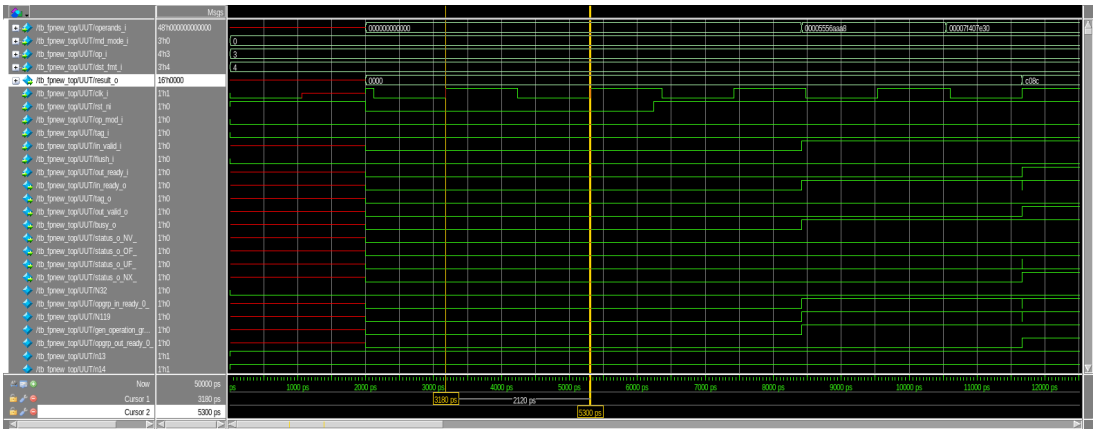

Figure 5: Netlist verification with Compile +Retiming

The clock cycle 2.12ns as the minimum clock period . When we simulate, we use this data as the time unit. After 10 multiplications, the time cycle is 21.2ns,see the figure6.

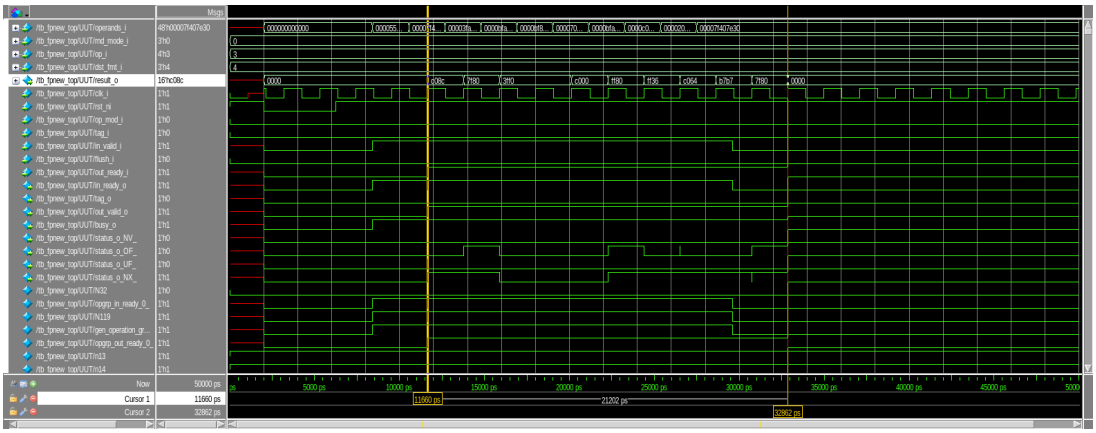

Figure 6: Netlist verification with 10 times multiplications

compile-ultra At third synthesis, we use 'compile-ultra' command. In one word, Design Ware is a set of reusable circuit-design blocks that intergrated in Synopsys, which means when we use this command, design compiler will automatically choose the suitable components with the best speed and area optimization from DW library to achieve the goal that reduce the clock period and area and finally improve the performance. From the table 2,we can see the clock period becomes 2.53ns ,the area become 3351.600018 µm2 and the maximum frequency is 395.26 MHz, see the table 3.The command file is in Appendix3.1.3, it shows the clock period and delay constraints, use compile-ultra command optimization, and generating netlist, delay and constraint files.

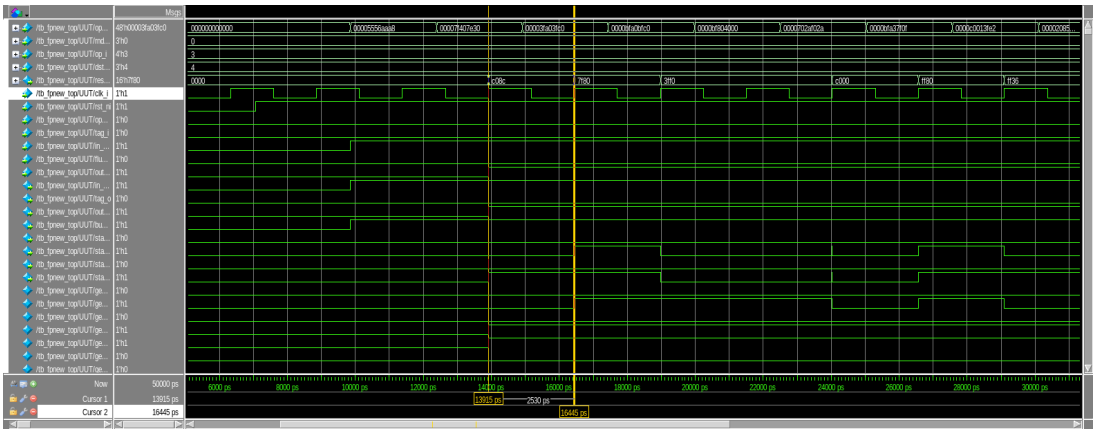

Figure 7: Netlist verification with Compile Ultra synthesis

The clock cycle 2.53ns as the minimum clock period . When we simulate, we use this data as the time unit. After 10 multiplications, the time cycle is 25.3ns,see the figure8

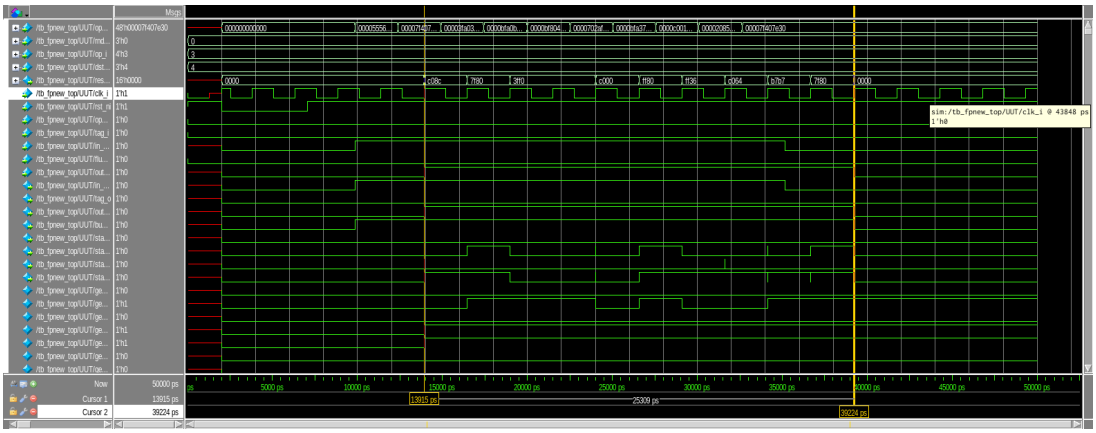

Figure 8: Netlist verification with 10 times multiplications

compile-CSA At fourth synthesis,we use 'compile' command with 'optimize-registers' and CSA(Carry Save Adder)architecture for multiplier deign. This method obviously let design compile use carry-save structure to achieve multiplication, which enhance the performance. CSA is an efficient architecture used in multipliers and dividers to perform addition operations. Instead of adding all bits immediately, some bits are stored as "Sum" and "Carry" and combined in later stages. This method reduces the critical path delay, making it suitable for high-speed circuits. // For implementation of CSA, we use command: *set_implementation DW02/csa [find cell* `*mult*`*]*. DW02 refers to a family of parameterized modules that implement arithmetic operations like multiplication, division, and square root. These modules are part of the DesignWare Foundation Library, which provides pre-optimized building blocks for commonly used functions in digital designs. The "DW" stands for DesignWare, while "02" is simply a numbering convention to group related arithmetic modules. `[find cell *mult*]` means find all cell about arithmetic operation multiplication, but in our case after compile in resource report(report refers to the resource utilization of the current design or selected parts of the design. This report provides information about the types and quantities of resources (e.g., instances of gates, flip-flops, DesignWare components) used in the synthesized netlist.), can be found only the cell

gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_form
_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/mult_325
its implementation setting is as shown in Fig 9.

```
Implementation Report
============================================================================
|                    |                    | Current            | Set            |
| Cell               | Module             | Implementation     | Implementation |
============================================================================
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_nu
|                    | DW01_inc           | pparch (area,speed)                 |
| add_1_root_add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_for
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_287 |
|                    | DW01_add           | pparch (area,speed)                 |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_nu
|                    | DW01_sub           | pparch (speed)     |                |
| add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_sl
add_368_2 |
|                    | DW01_add           | pparch (speed)     |                |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_nu
|                    | DW01_sub           | pparch (area,speed)                 |
| add_0_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_sl
add_515 |
|                    | DW01_inc           | rpl                |                |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_nu
|                    | DW02_mult          | csa                csa             |
============================================================================

1
```

Figure 9: Screenshot report resource csa

From the table 2, we can see the clock period becomes 2.31ns, the area become 3547.642039 µm2 and the maximum frequency is 432.90 MHz, see the table 3.The command file is in Appendix3.1.4, it contains commands to analyze and compile VHDL and Verilog files and set the min clock period and clock uncertainty. Resource command is3.2.3, it lists reuse of some components (eg.,adder and muliplier) in this design. The report detailed the parameters (eg.,width) and the specific operations in which these modules are called.

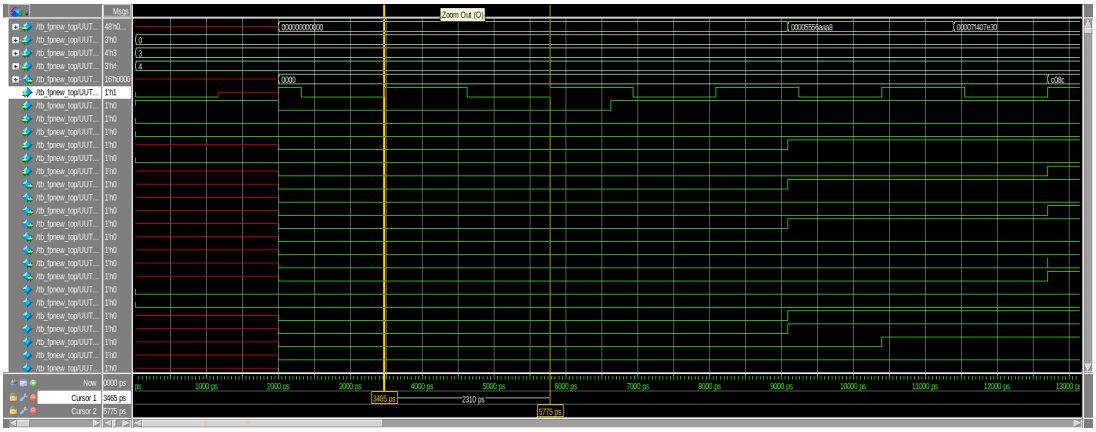

Figure 10: Netlist verification with Compile+CSA+Retiming

The clock cycle 2.31ns as the minimum clock period . When we simulate, we use this data

as the time unit. After 10 multiplications, the time cycle is 23.1ns,see the figure11

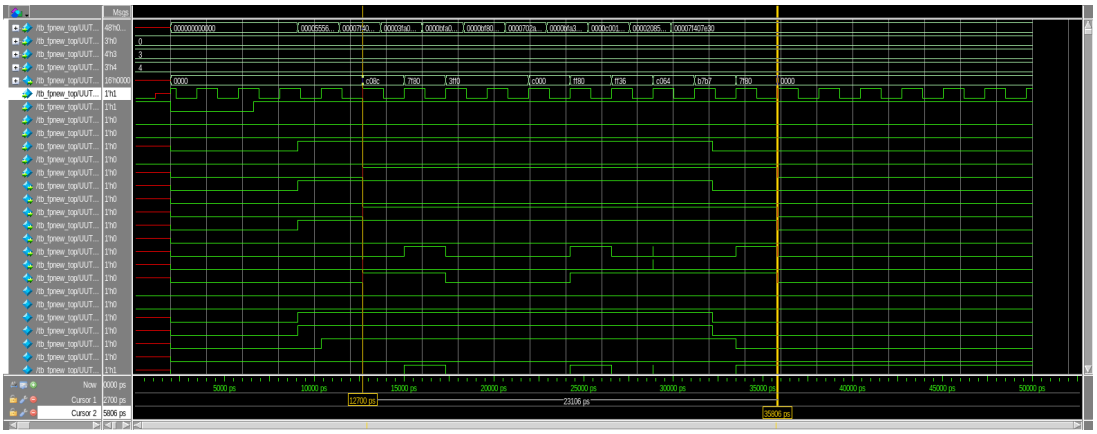

Figure 11: Netlist verification with 10 times multiplications

compile-PPARCH At fifth synthesis,we use 'compile' command with 'optimize-registers' and PPARCH(Parallel Prefix Architecture)architecture for multiplier design. This structure allowed parallel computation, which improve the critical path and reduce the overall latency.

PPARCH is an architecture used to implement adders with lower delays and faster performance compared to other methods.

It uses techniques like Brent-Kung, Kogge-Stone, and Sklansky to parallelize addition operations. The main idea is to propagate the Carry quickly using a tree structure, enabling parallel bit processing. From the table 2,we can see the clock period becomes 2.35ns ,the area become 3432.464039 µm2 and the maximum frequency is 425.53 MHz, see the table 3. The command file is in Appendix3.1.5, it contains commands to analyze and compile VHDL and Verilog files and set the min clock period and clock uncertainty. Resource command is 3.2.4, it shows the sharing and multiplexing of some components during synthesis, also detailed tell us the width of each components, their parameters and operations we used in the design.

As same as the previous section, in reource report, we find which cell is implemented approach pparch(fig12).

```
===============================================================================
|                          |                        | Current                | Set            |
| Cell                     | Module                 | Implementation         | Implementation |
===============================================================================
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_f
|                          | DW01_inc               | pparch (area,speed)                     |
| add_1_root_add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slice
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_287 |
|                          | DW01_add               | pparch (area,speed)                     |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_f
|                          | DW01_sub               | pparch (speed)         |                |
| add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active
add_368_2 |
|                          | DW01_add               | pparch (speed)         |                |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_f
|                          | DW01_sub               | pparch (area,speed)                     |
| add_0_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active
add_515 |
|                          | DW01_inc               | rpl                    |                |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_f
|                          | DW02_mult              | pparch (area)          | pparch         |
|                          |                        | mult_arch: and         |                |
===============================================================================

1
```

Figure 12: Screenshot report resource pparch

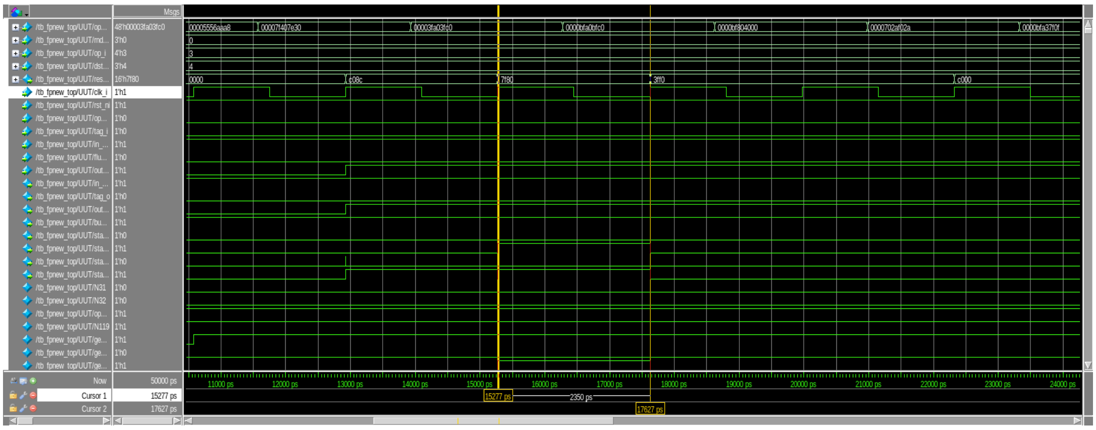

Figure 13: Netlist verification with Compile+PPARCH+Retiming

The clock cycle 2.35ns as the minimum clock period . When we simulate, we use this data as the time unit. After 10 multiplications, the time cycle is 23.5ns,see the figure14

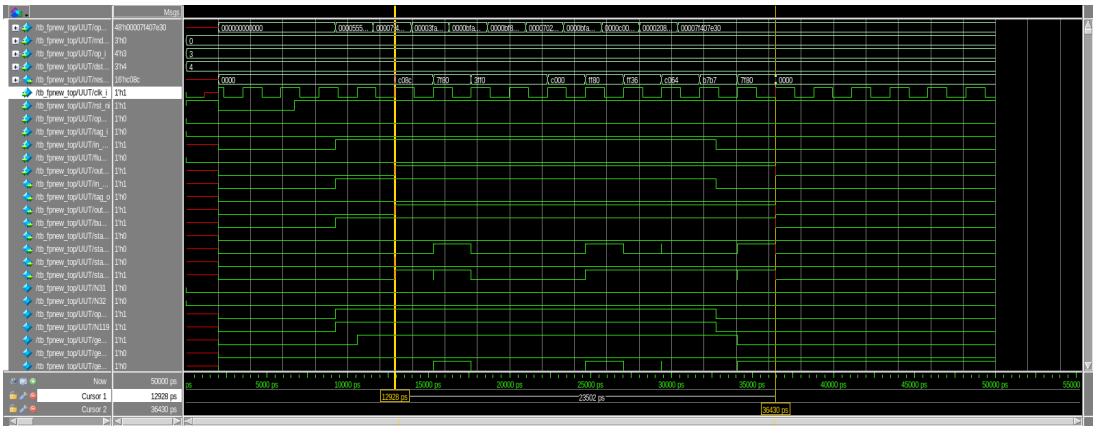

Figure 14: Netlist verification with 10 times multiplications

Syntesys type	Min CLK Period (ns)	Area (μm^2)
Compile	2.87	2937.438021
Compile+optimize registers	2.12	3634.358040
Compile Ultra	2.53	3351.600018
Compile+optimize registers+CSA	2.31	3547.642039
Compile+optimize registers+PPARCH	2.35	3432.464039

Table 2: Comparison of Synthesis Methods

Synthesis type	Maximum Frequency (MHz)
Compile	348.43
Compile + optimize registers	471.70
Compile Ultra	395.26
Compile + optimize registers + CSA	432.90
Compile + optimize registers + PPARCH	425.53

Table 3: Maximum Frequency for different Synthesis types.

1.4 Explanations, comparisons and comments

These five synthesis methods shows several optimization strategies for design performance and area. At start, the first command 'compile' achieves basic optimization for timing and area.
By adding 'optimize registers' in the second synthesis step, retiming is performed to improve the critical path, but at the cost of increased area, as show in Table 2, with Command single optimize_registers will generate an architecture with min clk period less than others but the area is the most between all synthesis type used.
The command 'compile ultra' further optimized the result by leveraging the DW components to achieve better balance between the speed and area. The compile_ultra command incorporates timing-focused optimization algorithms. The key algorithms used during the compilation process include:

1)Timing-driven high-level optimization Advanced optimization techniques are applied with timing as the primary focus. 2)Selection of appropriate macrocell structures for arithmetic operations This ensures optimal performance and area efficiency for arithmetic computations.

3)Choosing the best datapath implementations from the Design Ware library It intelligently selects the most suitable implementations for datapaths to meet design goals. 4)Mapping wide-fanin gates to reduce the number of logic levels Wide-fanin gates help improve timing by minimizing delays in the critical paths. 5)Proactive use of logic replication for load isolation Logic duplication is employed to separate heavily loaded circuits and improve performance. 6)Automatic ungrouping of hierarchies on critical paths Critical hierarchies are automatically flattened to improve timing and reduce delays.

However, from fourth and fifth synthesis ,we introduce the specialized architecture such as CSA and PPARCH to achieve specialized multiplier design . CSA enhance the performance by using Carry-save structure, while PPARCH increases parallelism to reduce latency. 'PPARCH'make the lowest latency with a suitable area successfully. In the Compile + optimize registers + CSA method, the Carry-Save Adder (CSA) architecture is used, which is designed to reduce computational time for adders. This architecture achieves faster computation by handling carry signals simultaneously and in parallel. However, due to the additional registers and logic required to manage these operations, it consumes more hardware resources, leading to a larger area.

On the other hand, in the Compile + optimize registers + PPARCH method, the Parallel-Prefix Architecture (PPARCH) is employed, which implements adders in a tree-like structure to perform computations in parallel. While this also reduces computation time, it uses hardware resources more efficiently compared to CSA, resulting in a smaller area. However, because of the tree-based structure, PPARCH introduces slightly more delay than CSA. In a summary, those methods both can up to the design requirement when the user choose them, according to different situation choose suitable way. Of course, they shows how the command-level optimization and structure can achieve customized design.

2 R4-MBE multiplier

In this section, we are required to optimized the mantissa multiplication using a R4-MBE multiplier based on dadda tree approach. The mantissa multiplication is found in *fpnew_fma.sv*: `product=mantissa_a* mantisssa_b`, we decided to design a module in System Verilog named `topmultiplier.sv` to replace it, as shown in figure 15 , it include 2 block: full booth recoding and dadda tree. Full booth recoding used R4-MBE approach to produce 5 operands and pass themselves and their sign to dadda tree block.

2.1 architecture

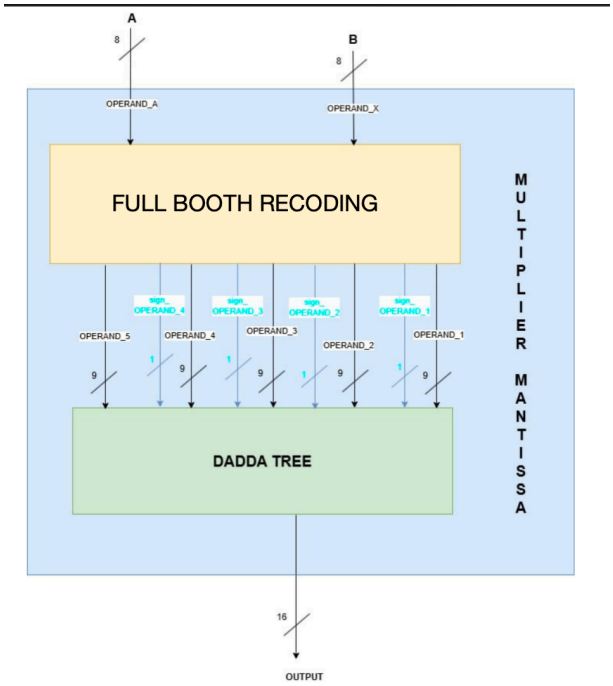

Figure 15: The whole simplified architecture of FPU

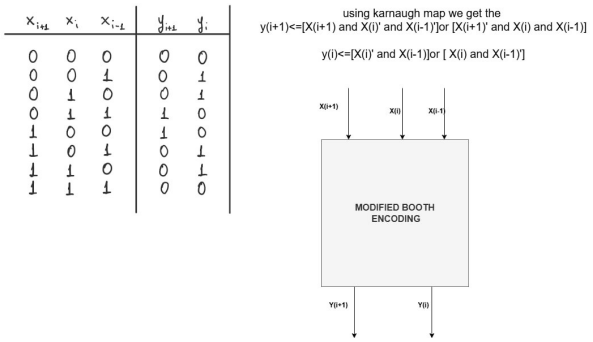

Figure 16: booth encoder component

2.1.1 full booth recoding

Under the block full booth recoding , there are 2 components: 6to1_multiplexer and modified booth encoding,as shown in figure 16, and 17.

In our case, we extend the `mantissa_b` from 8 bits to 11 bits by adding one zero to the tail and 2 bits to the head. The modified Booth encoding takes 3 bits at one time, starting from the least significant bit (e.g., `x[1] downto '0'`, `x[3] downto x[1]`, etc.) to generate a 2-bit encoded value.

This 2-bit encoded value, along with the most significant bit (MSB) of the three input bits, is used to select the correct operand in 6-to-1 multiplexer.

The multiplexer produces 6 possible operands, each 9 bits wide:

- `+0`: 9 bits of `'0'`

- `A:` 9 bits of `0&A`
- `2A:` 9 bits of `A&0`
- `A_inv:` 9 bits of `not(0&A)`
- `2A_inv:` 9 bits of `not(A&0)`
- `-0:` 9 bits of `'1'`

This process is illustrated in Table 4. Here, `A` represents `mantissa_a`.

2.1.2 dadda tree

As shown in Design 18, operands generated from upper significand bit will shifted 2 bit to left respect to previous one, it due to we used radix-4 MBE in previous part. **(1 s') (s' s s)** at beginning of the operands is used to resolve the overflow. operands generated from multiplexer should be represented in 2' complement, but we only invert the positive partial product to represent a negative partial product at 6to1 multiplexer, so at the dadda tree part , we need to add at the tail a bit using value of sign of each operand(if the operand is positive s='0', otherwise it equal to '1').
For dadda tree implementation, is required to use as late as possible the FA and HA and number of each layer from top to bottom should has 1.5 time than which in the under layer; at root layer we have always 2 operands, then at 2° layer we have 3 operands, at 4° there should be 4.5 operands(we decided to use 4 operands), finally at the top layer there are 5 operands(it is as same as the number of which one at the input). Our architecture will generate 17 bit as output, only the least significand 16 bits are required.

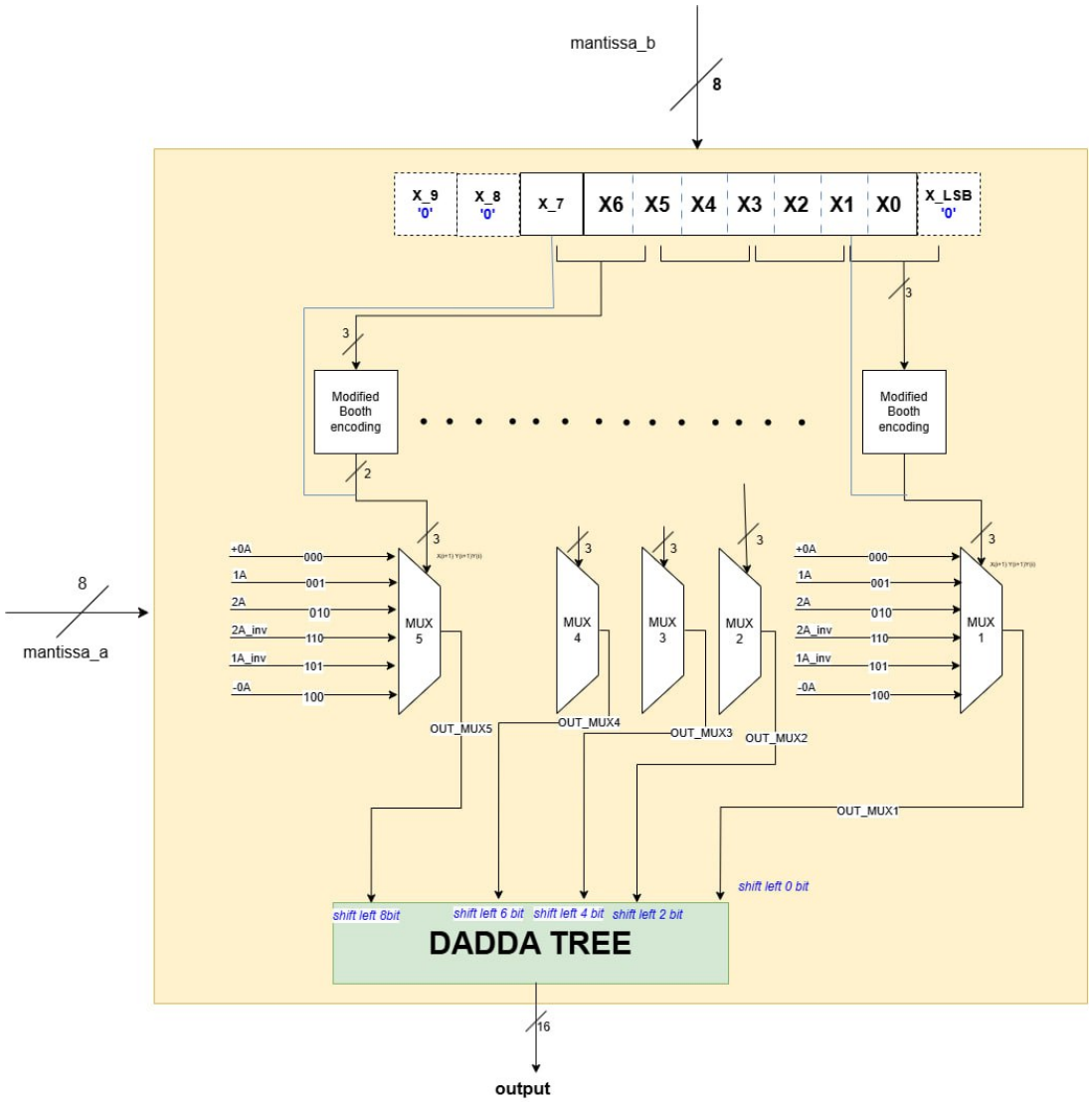

Figure 17: R4-MBE multiplier architecture

2.1.3 summary

Our simple structure diagram start with the booth encoder: Stand multiplier has been replaced by Radix-4 modified booth encoder multiplier, it can reduce the number of partial products significantly. To further enhance the performance, the partial products summation is achieved visa a 'Dadda Tree' structure. By combining these two techniques, whole architecture can handle overflow, underflow and other edge cases easily, ensuring the precision of arithmetic operation. From the figure 17,we implement an 8-bit multiplier with a 16-bit output.

traditional Binary	Booth's code	output
000	00	+0
001	01	A
010	01	2A
110	-01	2A_inv
101	-11	A_inv
100	-10	0_inv

Table 4: Booth's encoder trasformation

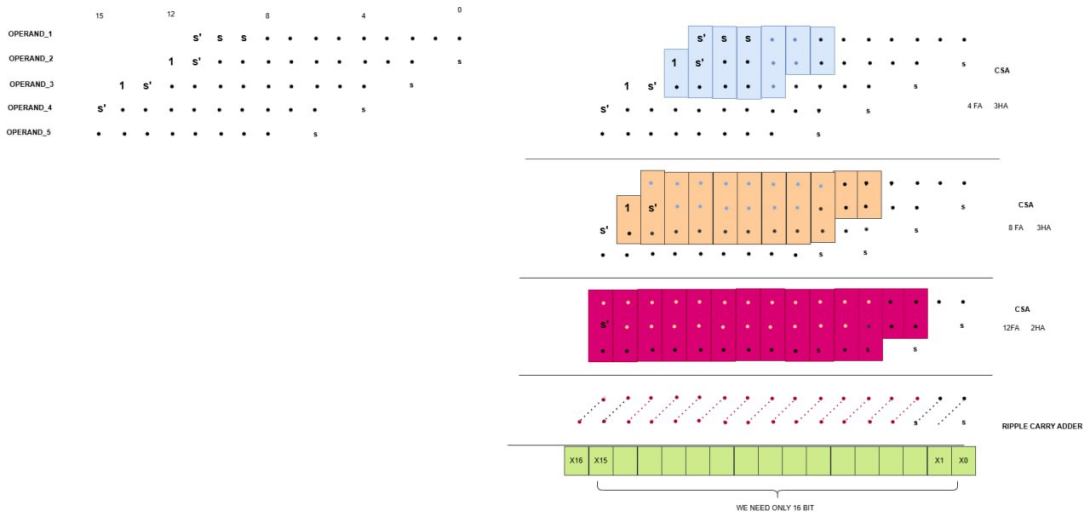

Figure 18: Dadda Tree with sign extension logic

2.1.4 advantages

The diagram highlights 3 compression stages, and these stages combined Full adders and Half adders to reduce the number of bits in different columns. In one word, Full adder reduces three bits to two while Half adder reduces two bits to one. The Dadda tree efficiently reduces partial products by systematically compressing the bit matrix in multiple stages. Each partial product from MUX is shifted and aligned to suitable position in the bit matrix. This alignment forms the input to the dadda tree structure. The dadda tree performs a stage reduction of the bit matrix, using FA and HA to reduce the rows that allowed in each stage to minimize the operands until there are only two rows left.This systematic compression reduce the computational complexity, once it remains the last two rows, at the root layer the Ripple Carry adder calculate the final sum, this is the output of two 8-bit operands.

2.2 Simulation

Standalone multiplier To verify the functionality of our multiplier as a stand-alone block,we decided to create a *tb_topmultiplier.v* to test our *top_multiplier.sv*. The result is in figure 19. The result confirms that the multiplier works correctly .

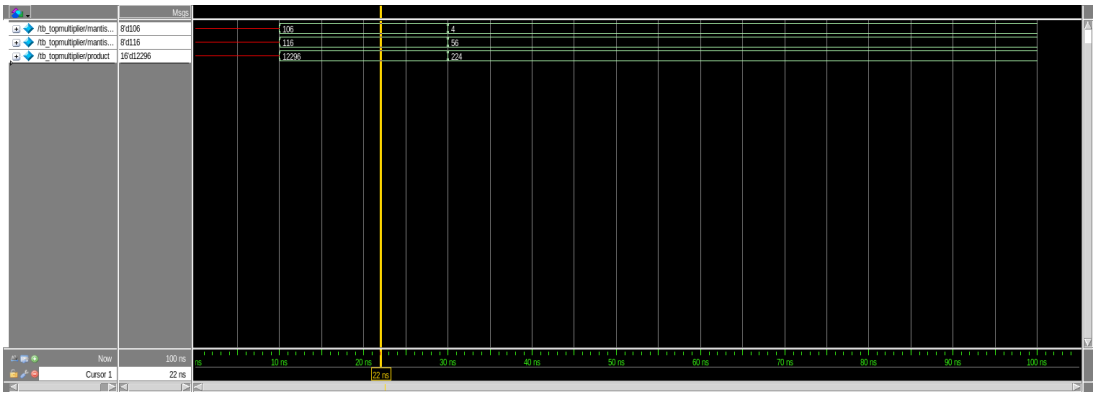

Figure 19: Simulation of the multiplication operation with the R4-MBE multiplier

2.3 Whole FPU

Once the multiplier was completed, it was inserted into the floating point unit as a component within the `fpnew_fma.sv` file and connect to `mantissa_a` and `mantissa_b` wires.The figure 20 shows the simulation with QuestaSim of the whole FPU with the new multiplier.The input value remains the same as in the previous simulation in **FPU unit**, and the result shows consistent outputs.

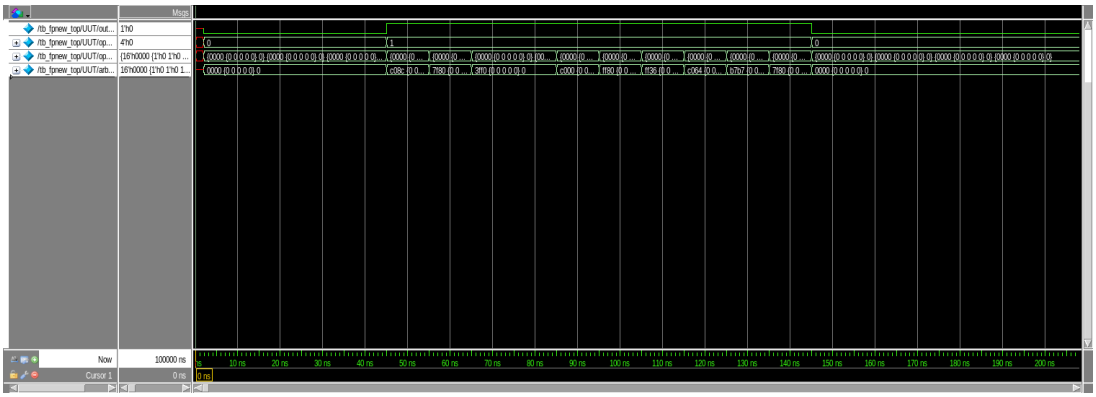

Figure 20: Simulation of the FPU multiplication operation

2.4 Synthesis

In the previous section, we have introduced in detail the specific role and function of each type of 'compile' instruction in the previous synthesis part. This part was redone separately as required by 'Compile Ultra', we can see the data of this command in table 5, for further details, we have upload the Area report in Appendix3.2.1, this report disaplay the number of ports,networks and cells of the circuit,the area for combinations and timing cells. The timing report in Appendix 3.2.2, it records the path delay, data arrival time and clock period, critical path is 2.51ns and the slack is 0. The commands about how to get them in Appendix 3.1.6, it include file analyze and compile process, we set the clock period and clock uncertainty. Timing and Area report are generated and finally the synthesized netlist file(.v) and timing constraint file(.sdc) are output.

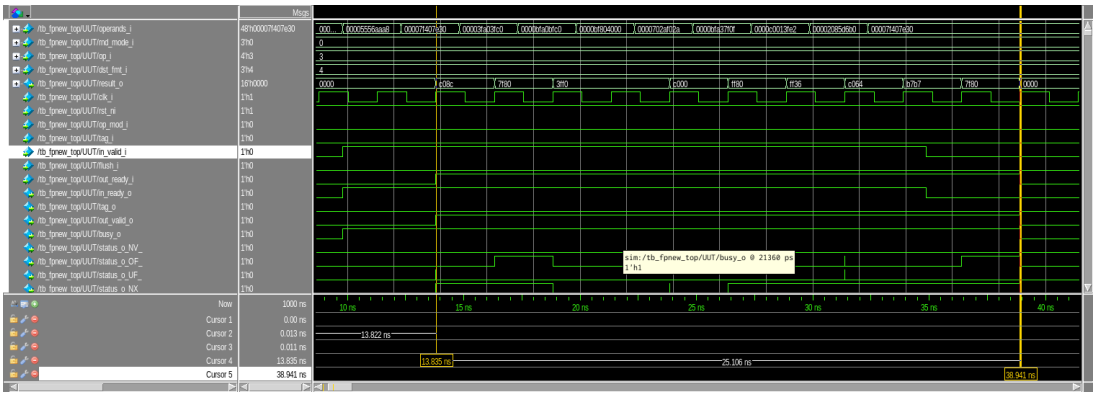

Figure 21: NEW netlist verification with Compile Ultra

Synthesis type	Maximum Frequency (MHz)	Area(μm^2)
Compile Ultra	398.42	3337.502016

Table 5: Maximum Frequency and Area for new compile ultra synthese.

2.5 Explanations, comparisons and comments

From the diagram, we can see the minim clock period has became 2.5106ns,after 10 times multiplications, it shows 25.106ns. Compared with previous one 2.53ns, we achieved a 0.0294ns improvement, which is enough to prove that our optimization was successful. It also shows that the dadda-tree structure reduces the complexity of the multiplication process, ensuring faster calculations with minimal resource overhead. The Dadda Tree approach is particularly advantageous in digital multipliers, where speed and efficiency are critical.

3 Appendix

3.1 commands

3.1.1 command_synthesis_compile.txt

```
source /eda/scripts/init_design_vision
design_vision
#dc_shell-xg-t
#  reaing source files;



analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/cf_math_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/lzc.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/rr_arb_tree.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_classifier.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_rounding.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_fma.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_fmt_slice.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_block.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_top.sv

#analyze -f vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/clk_gen.vhd
#analyze -f vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/data_gen16.vhd
#analyze -f sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_net.sv
#analyze -f sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_rtl.sv


set power_preserve_rtl_hier_names true

elaborate fpnew_top -lib WORK

uniquify

link
#  applying constraint

create_clock -name MY_CLK -period 2.87 clk_i
#create_clock -name MY_CLK -period 3.20 clk_i

set_dont_touch_network MY_CLK

set_clock_uncertainty 0.07 [get_clocks MY_CLK]

set_input_delay 0.5 -max -clock MY_CLK [remove_from_collection [all_inputs] clk_i]

set_output_delay 0.5 -max -clock MY_CLK [all_outputs]

ungroup -all -flatten

set OLOAD [load_of NangateOpenCellLibrary/BUF_X4/A]
set_load $OLOAD [all_outputs]

compile
#compile_ultra

#optimize_registers

report_resources >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile/resources_compile.txt
report_timing >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile/timing_compile.txt
report_area >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile/area_compile.txt



ungroup -all -flatten

#set_implementation DW02_mult/csa [find cell *mult*]



#  save the information for power estimation

change_names -hierarchy -rules sverilog

write_sdf /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile/netlist/fpnew_top.sdf



write -f verilog -hierarchy -output /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile/netlist/fpnew_top.v

write_sdc /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile/netlist/fpnew_top.sdc
quit
```

3.1.2 comand_synthesis_compile & retiming.txt

```
source /eda/scripts/init_design_vision
design_vision
#dc_shell-xg-t
#  reaing source files;



analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/cf_math_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/lzc.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/rr_arb_tree.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_classifier.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_rounding.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_fma.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_fmt_slice.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_block.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_top.sv

#analyze -f vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/clk_gen.vhd
#analyze -f vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/data_gen16.vhd
#analyze -f sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_net.sv
#analyze -f sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_rtl.sv


set power_preserve_rtl_hier_names true
#set_optimize_registers true
elaborate fpnew_top -lib WORK

uniquify

link
#  applying constraint

#create_clock -name MY_CLK -period 2.87 clk_i
create_clock -name MY_CLK -period 2.12 clk_i

set_dont_touch_network MY_CLK

set_clock_uncertainty 0.07 [get_clocks MY_CLK]

set_input_delay 0.5 -max -clock MY_CLK [remove_from_collection [all_inputs] clk_i]

set_output_delay 0.5 -max -clock MY_CLK [all_outputs]

ungroup -all -flatten

set OLOAD [load_of NangateOpenCellLibrary/BUF_X4/A]
set_load $OLOAD [all_outputs]

compile
#compile_ultra

optimize_registers

report_resources >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_retiming/resources_compile_retiming.txt
report_timing >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_retiming/timing_compile_retiming.txt
report_area >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_retiming/area_compile_retiming.txt



ungroup -all -flatten




#set_implementation DW02_mult/csa [find cell *mult*]


#  save the information for power estimation

change_names -hierarchy -rules sverilog

write_sdf /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_retiming/netlist/fpnew_top.sdf

write -f verilog -hierarchy -output /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_retiming/netlist/fpnew_top.v

write_sdc /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_retiming/netlist/fpnew_top.sdc
```

3.1.3 command_synthesis_compile_ultra.txt

```
source /eda/scripts/init_design_vision
design_vision
#dc_shell-xg-t
#  reaing source files;



analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/cf_math_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/lzc.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/rr_arb_tree.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_classifier.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_rounding.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_fma.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_fmt_slice.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_block.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_top.sv

#analyze -f vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/clk_gen.vhd
#analyze -f vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/data_gen16.vhd
#analyze -f sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_net.sv
#analyze -f sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_rtl.sv


set power_preserve_rtl_hier_names true

elaborate fpnew_top -lib WORK

uniquify

link
#  applying constraint

create_clock -name MY_CLK -period 2.53 clk_i
#create_clock -name MY_CLK -period 3.20 clk_i

set_dont_touch_network MY_CLK

set_clock_uncertainty 0.07 [get_clocks MY_CLK]

set_input_delay 0.5 -max -clock MY_CLK [remove_from_collection [all_inputs] clk_i]

set_output_delay 0.5 -max -clock MY_CLK [all_outputs]

ungroup -all -flatten

set OLOAD [load_of NangateOpenCellLibrary/BUF_X4/A]
set_load $OLOAD [all_outputs]

#compile
compile_ultra

#optimize_registers

report_resources >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_ultra/resources_compile_ultra.txt
report_timing >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_ultra/timing_compile_ultra.txt
report_area >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_ultra/area_compile_ultra.txt



ungroup -all -flatten
#set_implementation DW02_mult/csa [find cell *mult*]



#  save the information for power estimation

change_names -hierarchy -rules sverilog

write_sdf /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_ultra/netlist/fpnew_top.sdf



write -f verilog -hierarchy -output /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_ultra/netlist/fpnew_top.v

write_sdc /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_ultra/netlist/fpnew_top.sdc
quit
```

3.1.4 command_synthesis_compile_csa.txt

```
source /eda/scripts/init_design_vision
design_vision
#dc_shell-xg-t
#  reaing source files;



analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/cf_math_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/lzc.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/rr_arb_tree.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_classifier.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_rounding.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_fma.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_fmt_slice.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_block.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_top.sv

#analyze -f vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/clk_gen.vhd
#analyze -f vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/data_gen16.vhd
#analyze -f sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_net.sv
#analyze -f sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_rtl.sv


set power_preserve_rtl_hier_names true

elaborate fpnew_top -lib WORK

uniquify

link
#  applying constraint

create_clock -name MY_CLK -period 2.31 clk_i
#create_clock -name MY_CLK -period 3.20 clk_i

set_dont_touch_network MY_CLK

set_clock_uncertainty 0.07 [get_clocks MY_CLK]

set_input_delay 0.5 -max -clock MY_CLK [remove_from_collection [all_inputs] clk_i]

set_output_delay 0.5 -max -clock MY_CLK [all_outputs]

ungroup -all -flatten

set OLOAD [load_of NangateOpenCellLibrary/BUF_X4/A]
set_load $OLOAD [all_outputs]

set_implementation DW02_mult/csa [find cell *mult*]
compile
#compile_ultra

optimize_registers




report_resources >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_csa/resources_compile.txt
report_timing >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_csa/timing_compile.txt
report_area >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_csa/area_compile.txt


#  save the information for power estimation
ungroup -all -flatten

change_names -hierarchy -rules sverilog

write_sdf /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_csa/netlist/fpnew_top.sdf



write -f verilog -hierarchy -output /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_csa/netlist/fpnew_top.v

write_sdc /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_csa/netlist/fpnew_top.sdc
quit
```

3.1.5 command_pparch.txt

```
source /eda/scripts/init_design_vision
design_vision
#dc_shell-xg-t
#  reaing source files;



analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/cf_math_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/lzc.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/rr_arb_tree.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_classifier.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_rounding.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_fma.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_fmt_slice.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_block.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_top.sv

#analyze -f vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/clk_gen.vhd
#analyze -f vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/data_gen16.vhd
#analyze -f sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_net.sv
#analyze -f sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_rtl.sv


set power_preserve_rtl_hier_names true

elaborate fpnew_top -lib WORK

uniquify

link
#  applying constraint

create_clock -name MY_CLK -period 2.35 clk_i
#create_clock -name MY_CLK -period 3.20 clk_i

set_dont_touch_network MY_CLK

set_clock_uncertainty 0.07 [get_clocks MY_CLK]

set_input_delay 0.5 -max -clock MY_CLK [remove_from_collection [all_inputs] clk_i]

set_output_delay 0.5 -max -clock MY_CLK [all_outputs]

ungroup -all -flatten

set OLOAD [load_of NangateOpenCellLibrary/BUF_X4/A]
set_load $OLOAD [all_outputs]

set_implementation DW02_mult/pparch [find cell *mult*]
compile
#compile_ultra

optimize_registers




report_resources >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_pparch/resources_compile.txt
report_timing >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_pparch/timing_compile.txt
report_area >> /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_pparch/area_compile.txt


#  save the information for power estimation
ungroup -all -flatten

change_names -hierarchy -rules sverilog

write_sdf /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_pparch/netlist/fpnew_top.sdf



write -f verilog -hierarchy -output /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_pparch/netlist/fpnew_top.v

write_sdc /home/isa05_2024_2025/Desktop/monica/lab02/syn/compile_pparch/netlist/fpnew_top.sdc
quit
```

3.1.6 command_topmultiplier_synthesis.txt

```
source /eda/scripts/init_design_vision
design_vision
#dc_shell-xg-t
#  reaing source files;



analyze -f  vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/src/booth_radix4.vhd
analyze -f  vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/src/full_adder.vhd
analyze -f  vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/src/half_adder.vhd
analyze -f  vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/src/mult6to1.vhd
analyze -f  vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/src/full_booth.vhd
analyze -f  vhdl -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/src/dadda_tree.vhd
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/src/topmultiplier.sv


analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/cf_math_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/lzc.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/rr_arb_tree.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_pkg.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_classifier.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_rounding.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/src/fpnew_fma.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_fmt_slice.sv
analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_opgroup_block.sv

analyze -f  sverilog -lib WORK /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_top.sv


#vlog -work ./work /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_net.sv
#vlog -work ./work /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/tb/tb_fpnew_top_rtl.sv



set power_preserve_rtl_hier_names true

elaborate fpnew_top -lib WORK

uniquify

link
#  applying constraint

create_clock -name MY_CLK -period 2.51 clk_i
#create_clock -name MY_CLK -period 3.20 clk_i

set_dont_touch_network MY_CLK

set_clock_uncertainty 0.07 [get_clocks MY_CLK]

set_input_delay 0.5 -max -clock MY_CLK [remove_from_collection [all_inputs] clk_i]

set_output_delay 0.5 -max -clock MY_CLK [all_outputs]

ungroup -all -flatten

set OLOAD [load_of NangateOpenCellLibrary/BUF_X4/A]
set_load $OLOAD [all_outputs]

#compile
compile_ultra

#optimize_registers

report_timing >> /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/syn/report/timing_compile_ultra.txt
report_resources >> /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/syn/report/resources_compile_ultra.txt

report_area >> /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/syn/report/area_compile_ultra.txt



ungroup -all -flatten
#set_implementation DW02_mult/csa [find cell *mult*]



#  save the information for power estimation

change_names -hierarchy -rules sverilog

write_sdf /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/syn/netlist/fpnew_top.sdf



write -f verilog -hierarchy -output /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/syn/netlist/fpnew_top.v

write_sdc /home/isa05_2024_2025/Desktop/monica/lab02/mbe_multip/syn/netlist/fpnew_top.sdc

quit
```

3.2 report

3.2.1 topmultiplier_area_ultra_report.txt

```
****************************************
Report : area
Design : fpnew_top
Version: S-2021.06-SP4
Date   : Fri Dec 20 14:18:18 2024
****************************************

Library(s) Used:

    NangateOpenCellLibrary (File: /eda/dk/nangate45/synopsys/NangateOpenCellLibrary_typical_ecsm.db)

Number of ports:                         96
Number of nets:                        2842
Number of cells:                       2639
Number of combinational cells:         2505
Number of sequential cells:             132
Number of macros/black boxes:             0
Number of buf/inv:                      428
Number of references:                    45

Combinational area:              2634.197994
Buf/Inv area:                     241.528001
Noncombinational area:            703.304022
Macro/Black Box area:               0.000000
Net Interconnect area:      undefined  (Wire load has zero net area)

Total cell area:                 3337.502016
Total area:                 undefined
1
```

3.2.2 topmultiplier_timing_ultra_report.txt

```
Report : timing
        -path full
        -delay max
        -max_paths 1
Design : fpnew_top
Version: S-2021.06-SP4
Date   : Fri Dec 20 14:18:38 2024
****************************************
Operating Conditions: typical   Library: NangateOpenCellLibrary
Wire Load Model Mode: top

  Startpoint: gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_num_lanes[0].active_lane.
  lane_instance.i_fma/inp_pipe_operands_q_reg[1][0][5]
              (rising edge-triggered flip-flop clocked by MY_CLK)
  Endpoint: gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_num_lanes[0].active_lane.
  lane_instance.i_fma/mid_pipe_sum_q_reg[1][19]
            (rising edge-triggered flip-flop clocked by MY_CLK)
  Path Group: MY_CLK
  Path Type: max

  Des/Clust/Port     Wire Load Model       Library
  ------------------------------------------------
  fpnew_top          5K_hvratio_1_1        NangateOpenCellLibrary
  Point                                                 Incr       Path
  ---------------------------------------------------------------------------
  clock MY_CLK (rise edge)                              0.00       0.00
  clock network delay (ideal)                           0.00       0.00
  gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.
  i_fma/inp_pipe_operands_q_reg[1][0][5]/CK (DFFR_X1)
                                                        0.00       0.00 r
  gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.
  i_fma/inp_pipe_operands_q_reg[1][0][5]/QN (DFFR_X1)
                                                        0.07       0.07 r
  U2211/ZN (NAND4_X1)                                   0.04       0.11 f
  U1162/ZN (OR4_X1)                                     0.13       0.24 f
  U1238/ZN (NOR2_X1)                                    0.06       0.30 r
  U2212/ZN (AND2_X1)                                    0.05       0.35 r
  U1924/ZN (OR2_X2)                                     0.04       0.39 r
  U2227/ZN (INV_X1)                                     0.03       0.43 f
  U1755/ZN (NAND2_X1)                                   0.03       0.46 r
  U1660/ZN (NAND2_X1)                                   0.04       0.50 f
  U1276/ZN (NAND2_X1)                                   0.03       0.53 r
  U1277/ZN (NAND3_X1)                                   0.05       0.58 f
  U1746/ZN (NAND2_X1)                                   0.04       0.61 r
  U1748/ZN (NAND3_X1)                                   0.05       0.66 f
  U1281/ZN (NAND2_X1)                                   0.04       0.69 r
  U1283/ZN (NAND3_X1)                                   0.05       0.74 f
  U1734/ZN (NAND2_X1)                                   0.04       0.78 r
  U1736/ZN (NAND3_X1)                                   0.05       0.82 f
  U1740/ZN (NAND2_X1)                                   0.04       0.86 r
  U1742/ZN (NAND3_X1)                                   0.04       0.89 f
  U2313/CO (FA_X1)                                      0.09       0.98 f
  U2312/CO (FA_X1)                                      0.09       1.07 f
  U2315/S (FA_X1)                                       0.14       1.22 r
  U1980/ZN (NAND4_X1)                                   0.05       1.27 f
  U1963/ZN (NAND2_X2)                                   0.06       1.33 r
  U1221/ZN (AND3_X1)                                    0.10       1.43 r
  U1861/ZN (OR3_X1)                                     0.06       1.49 r
  U2017/ZN (NAND2_X1)                                   0.03       1.52 f
  U1856/Z (MUX2_X1)                                     0.07       1.59 f
  U2426/ZN (AOI22_X1)                                   0.08       1.66 r
  U1890/ZN (OAI21_X1)                                   0.04       1.71 f
  U1803/ZN (XNOR2_X1)                                   0.08       1.79 r
  U2512/CO (HA_X1)                                      0.07       1.86 r
  U1388/ZN (AND2_X1)                                    0.05       1.91 r
  U1994/ZN (INV_X1)                                     0.02       1.93 f
  U1993/ZN (AND2_X1)                                    0.04       1.97 f
  U1171/ZN (INV_X1)                                     0.03       2.00 r
  U1991/ZN (NAND4_X1)                                   0.04       2.04 f
  U1316/ZN (AND3_X1)                                    0.05       2.09 f
  U1291/ZN (OAI211_X1)                                  0.04       2.13 r
  U2054/ZN (NAND2_X1)                                   0.03       2.16 f
  U1951/ZN (AND2_X2)                                    0.05       2.21 f
  U2035/ZN (OAI21_X1)                                   0.06       2.27 r
  U2075/ZN (XNOR2_X1)                                   0.06       2.33 r
  U2074/ZN (NAND2_X1)                                   0.03       2.36 f
  U3299/ZN (OAI211_X1)                                  0.03       2.39 r
  gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
  gen_num_lanes[0].active_lane.lane_instance.i_fma/mid_pipe_sum_q_reg[1][19]/D (DFFR_X1)
                                                        0.01       2.40 r
  data arrival time                                                2.40
  clock MY_CLK (rise edge)                              2.51       2.51
  clock network delay (ideal)                           0.00       2.51
  clock uncertainty                                    -0.07       2.44  gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
  i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/mid_pipe_sum_q_reg[1][19]/CK (DFFR_X1)
                                                        0.00       2.44 r
  library setup time                                   -0.04       2.40
  data required time                                               2.40
  ---------------------------------------------------------------------------
  data required time                                               2.40
  data arrival time                                               -2.40
  ---------------------------------------------------------------------------
  slack (MET)                                                      0.00
```

3.2.3 csa_resources_compile.txt

```
****************************************
Report : resources
Design : fpnew_top
Version: S-2021.06-SP4
Date   : Fri Dec  6 21:41:28 2024
****************************************

Resource Sharing Report for design fpnew_top in file
        /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_top.sv

===============================================================================
|          |              |            | Contained     |                      |
| Resource | Module       | Parameters | Resources     | Contained Operations |
===============================================================================
| r426     | DW01_cmp2    | width=3    |               | gen_operation_groups[0].i_opgroup_block/i_arbiter/gt_208_G4 |
|          |              |            |               |
|          |              |            |               | gen_operation_groups[0].i_opgroup_block/i_arbiter/lte_209_G4 |
| r427     | DW01_cmp2    | width=3    |               | gen_operation_groups[1].i_opgroup_block/i_arbiter/gt_208_G4 |
|          |              |            |               |
|          |              |            |               | gen_operation_groups[1].i_opgroup_block/i_arbiter/lte_209_G4 |
| r428     | DW01_cmp2    | width=3    |               | gen_operation_groups[2].i_opgroup_block/i_arbiter/gt_208_G4 |
|          |              |            |               |
|          |              |            |               | gen_operation_groups[2].i_opgroup_block/i_arbiter/lte_209_G4 |
| r429     | DW01_cmp2    | width=3    |               | gen_operation_groups[3].i_opgroup_block/i_arbiter/gt_208_G4 |
|          |              |            |               |
|          |              |            |               | gen_operation_groups[3].i_opgroup_block/i_arbiter/lte_209_G4 |
| r430     | DW01_cmp2    | width=2    |               | i_arbiter/gt_208_G4  |
|          |              |            |               | i_arbiter/lte_209_G4 |
| r460     | DW01_add     | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_285 |}
| r468     | DW01_sub     | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_293 |
| r470     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/gt_295 |
| r472     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/lte_302 |
| r474     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/lte_305 |
| r476     | DW01_sub     | width=5    |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_306 |
| r478     | DW02_mult    | A_width=8  |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/mult_325 |
|
|          |              | B_width=8  |               |                      |
| r480     | DW_rightsh   | A_width=36 |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/srl_349 |
|
|          |              | SH_width=5 |               |                      |
| r482     | DW01_add     | width=29   |               | add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_368_2 |
| r484     | DW01_sub     | width=28   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_372 |
| r486     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/lte_510 |
| r488     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/lte_510_2 |
| r494     | DW_cmp       | width=12   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/gte_512 |
| r496     | DW01_add     | width=5    |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_514 |
| r502     | DW01_add     | width=5    |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_519 |
| r504     | DW_leftsh    | A_width=29 |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sll_530 |
|
|          |              | SH_width=5 |               |                      |
| r506     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/gt_547 |
| r508     | DW01_inc     | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_542 |
| r510     | DW01_dec     | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_549 |
| r512     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/gte_576 |
| r514     | DW01_add     | width=15   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/i_fpnew_rounding/add_63 |
| r1242    | DW01_add     | width=10   |               | add_1_root_add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].
active_format.i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_287 |
| r1244    | DW01_sub     | width=10   |               | sub_2_root_add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].
active_format.i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_287 |
| r1246    | DW01_add     | width=10   |               | add_0_root_add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].
active_format.i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_287 |
| r1966    | DW01_sub     | width=11   |               | sub_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_512 |
| r1968    | DW01_inc     | width=12   |               | add_0_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_512 |
| r2688    | DW01_sub     | width=10   |               | sub_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_515 |
| r2690    | DW01_inc     | width=10   |               | add_0_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_515 |
===============================================================================
```

```
Implementation Report
============================================================================
|                    |                  | Current           | Set           |
| Cell               | Module           | Implementation    | Implementation |
============================================================================
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_542 |
|                    | DW01_inc         | pparch (area,speed)               |
| add_1_root_add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_287 |
|                    | DW01_add         | pparch (area,speed)               |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_num_lanes[0].
active_lane.lane_instance.i_fma/sub_372 |
|                    | DW01_sub         | pparch (speed)    |               |
| add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_368_2 |
|                    | DW01_add         | pparch (speed)    |               |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_num_lanes[0].
active_lane.lane_instance.i_fma/sub_293 |
|                    | DW01_sub         | pparch (area,speed)               |
| add_0_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_515 |
|                    | DW01_inc         | rpl               |               |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/gen_num_lanes[0].
active_lane.lane_instance.i_fma/mult_325 |
|                    | DW02_mult        | csa               | csa           |
============================================================================

1
```

3.2.4 pparch_resources_compile.txt

```
****************************************
Report : resources
Design : fpnew_top
Version: S-2021.06-SP4
Date   : Sun Dec  8 14:58:44 2024
****************************************

Resource Sharing Report for design fpnew_top in file
        /home/isa05_2024_2025/Desktop/monica/lab02/src/cvfpu_lite/src/fpnew_top.sv

===============================================================================
|          |              |            | Contained     |                      |
| Resource | Module       | Parameters | Resources     | Contained Operations |
===============================================================================
| r427     | DW01_cmp2    | width=3    |               | gen_operation_groups[0].i_opgroup_block/i_arbiter/gt_208_G4 |
|          |              |            |               |
|          |              |            |               | gen_operation_groups[0].i_opgroup_block/i_arbiter/lte_209_G4 |
| r428     | DW01_cmp2    | width=3    |               | gen_operation_groups[1].i_opgroup_block/i_arbiter/gt_208_G4 |
|          |              |            |               |
|          |              |            |               | gen_operation_groups[1].i_opgroup_block/i_arbiter/lte_209_G4 |
| r429     | DW01_cmp2    | width=3    |               | gen_operation_groups[2].i_opgroup_block/i_arbiter/gt_208_G4 |
|          |              |            |               |
|          |              |            |               | gen_operation_groups[2].i_opgroup_block/i_arbiter/lte_209_G4 |
| r430     | DW01_cmp2    | width=3    |               | gen_operation_groups[3].i_opgroup_block/i_arbiter/gt_208_G4 |
|          |              |            |               |
|          |              |            |               | gen_operation_groups[3].i_opgroup_block/i_arbiter/lte_209_G4 |
| r431     | DW01_cmp2    | width=2    |               | i_arbiter/gt_208_G4  |
|          |              |            |               | i_arbiter/lte_209_G4 |
| r461     | DW01_add     | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_285 |
| r469     | DW01_sub     | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_293 |
| r471     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/gt_295 |
| r473     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/lte_302 |
| r475     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/lte_305 |
| r477     | DW01_sub     | width=5    |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_306 |
| r479     | DW02_mult    | A_width=8  |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/mult_325 |
|
|          |              | B_width=8  |               |                      |
| r481     | DW_rightsh   | A_width=36 |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/srl_349 |
|
|          |              | SH_width=5 |               |                      |
| r483     | DW01_add     | width=29   |               | add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_368_2 |
| r485     | DW01_sub     | width=28   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_372 |
| r487     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/lte_510 |
| r489     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/lte_510_2 |
| r495     | DW_cmp       | width=12   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/gte_512 |
| r497     | DW01_add     | width=5    |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_514 |
| r503     | DW01_add     | width=5    |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_519 |
| r505     | DW_leftsh    | A_width=29 |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sll_530 |
|
|          |              | SH_width=5 |               |                      |
| r507     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/gt_547 |
| r509     | DW01_inc     | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_542 |
| r511     | DW01_dec     | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_549 |
| r513     | DW_cmp       | width=10   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/gte_576 |
| r515     | DW01_add     | width=15   |               | gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/i_fpnew_rounding/add_63 |
| r1243    | DW01_add     | width=10   |               | add_1_root_add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].
active_format.i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_287 |
| r1245    | DW01_sub     | width=10   |               | sub_2_root_add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].
active_format.i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_287 |
| r1247    | DW01_add     | width=10   |               | add_0_root_add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].
active_format.i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_287 |
| r1967    | DW01_sub     | width=11   |               | sub_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_512 |
| r1969    | DW01_inc     | width=12   |               | add_0_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_512 |
| r2689    | DW01_sub     | width=10   |               | sub_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_515 |
| r2691    | DW01_inc     | width=10   |               | add_0_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_515 |
===============================================================================
```

```
Implementation Report
============================================================================
|                    |                  | Current            | Set            |
| Cell               | Module           | Implementation     | Implementation |
============================================================================
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/add_542 |
|                    | DW01_inc         | pparch (area,speed)                |
| add_1_root_add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].
active_format.i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_287 |
|                    | DW01_add         | pparch (area,speed)                |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_372 |
|                    | DW01_sub         | pparch (speed)     |                |
| add_1_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_368_2 |
|                    | DW01_add         | pparch (speed)     |                |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/sub_293 |
|                    | DW01_sub         | pparch (area,speed)                |
| add_0_root_gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.
i_fmt_slice/gen_num_lanes[0].active_lane.lane_instance.i_fma/add_515 |
|                    | DW01_inc         | rpl                |                |
| gen_operation_groups[0].i_opgroup_block/gen_parallel_slices[4].active_format.i_fmt_slice/
gen_num_lanes[0].active_lane.lane_instance.i_fma/mult_325 |
|                    | DW02_mult        | pparch (area)      | pparch         |
|                    |                  | mult_arch: and     |                |
============================================================================

1
```